

Computación Paralela y Distribuida

2026-II

José Fiestas PhD

31 de agosto de 2026

Universidad de Ingeniería y Tecnología
jfiestas@utec.edu.pe



Objetivos:

1. Velocidad, eficiencia, escalabilidad. Ley de Ahmdal
2. Speedup, performance
3. Modelos computacionales en paralelo - PRAM (1)
4. Modelos computacionales en paralelo - PRAM (2)
5. **Particionamiento de datos y tareas**
6. **Random en paralelo**
7. N-body, Divide y Vencerás (mergesort)

U2.5 Particionamiento

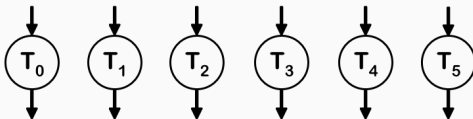


Particionamiento de Datos

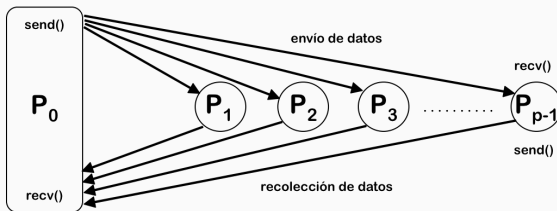
Particionamiento de datos: Cómputo

Donde las tareas son completamente separables.

El tiempo de cómputo (T_{comp}) puede ser totalmente independiente entre procesos, y depende del problema (secuencial) a optimizar



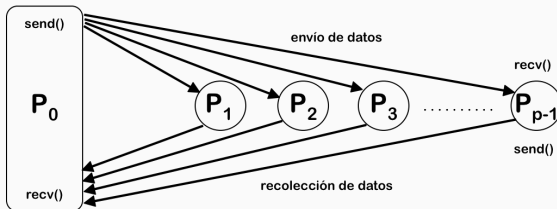
Particionamiento de datos: Comunicación



La comunicación se necesita para:

- Realizar el **particionamiento** desde el maestro
- De ser necesario, comunicación **entre procesos** previa al cálculo (e.g. suavizado) o **sincronización**
- Realizar **reducción/gather** final en el maestro

Particionamiento de datos: Comunicación



El **tiempo de comunicación** tiene los siguientes componentes

$$T_{comm} = f(p)(\alpha + x\beta)$$

Donde α es la **latencia** (tiempo invertido en enviar un mensaje de tamaño cero), y β es el **ancho de banda** (tiempo invertido en enviar un mensaje de un Byte)

El **factor $f(p)$** depende de la topología usada (distribución de procesos)

Ejemplo: suma de elementos de un array

El cálculo secuencial necesita $n-1$ adiciones, con complejidad $O(n)$.

Con p procesos se obtiene:

- Envío de data $T_{comm_1} = O(p(\alpha + (n/p)\beta))$
- Cálculo de cada proceso $T_{comp_1} = O(n/p)$
- Recolección de sumas parciales $T_{comm_2} = O(p(\alpha + \beta))$
- Suma total $T_{comp_2} = O(p)$

Complejidad es $T_p(n, p) = O(p\alpha + (n + p)\beta + n/p)$

Note que para evaluar la dominancia de términos en la expresión debe considerar el orden de magnitud de los parámetros α, β ($10^{-7} - 10^{-9}$)

En este ejemplo se utiliza una **topología lineal** $O(p)$.

Ejemplo: suma de elementos de un array

Con comunicación optimizada, podemos lograr un factor $O(\log p)$, que deriva en:

- Envío de data $T_{comm_1} = O(\alpha \log p + (n/p)\beta)$
- Cálculo de cada proceso $T_{comp_1} = O(n/p)$
- Reducción de sumas parciales $T_{comm_2} = O(\log p(\alpha + \beta + 1))$

Complejidad es $T_p(n, p) = O(n/p + \alpha \log p + (n/p)\beta)$

Note que el span $T_\infty(n) = O(\log n)$ se convierte en el factor de comunicación $\log p$, debido al calculo local de cada procesador, cuyo resultado es un elemento que se propaga y acumula en el árbol de profundidad $\log p$.

Un array lineal con p procesos se representa por un grafo $G(V,E)$, todos tienen un vecino anterior y posterior, con excepción del primero y el último.

Para un **broadcast simple**, el maestro envía el mensaje a sus vecinos, y estos, a su vez, a los propios. Tiene un tiempo asintótico

$$T_{bcast} = O(p(\alpha + m\beta))$$

Esto depende de la posición del maestro. Si está al final del array, serían $p-1$ pasos, en el medio tendría $\lfloor p/2 \rfloor$

Un **particionamiento de m datos** se ejecuta en un tiempo

$$T_{scatter/gather} \approx (p-1)(\alpha + \beta \frac{m}{p}) = \alpha(p-1) + \beta m(1 - 1/p), \text{ i.e.}$$

$$T_{scatter/gather} = O(\alpha p + \beta m)$$

Comunicación en árbol (binario)

Si p procesadores estean dispuestos en los nodos de un árbol binario, el **broadcast** de m datos se ejecuta en un tiempo

$$T_{bcast} = O(\log p(\alpha + m\beta))$$

Para un **particionamiento de m datos** los envíos punto a punto se organizan en $\log p$ niveles. En el nivel i , el tamaño del mensaje acumulado es $2^i \frac{m}{p}$. Por tanto:

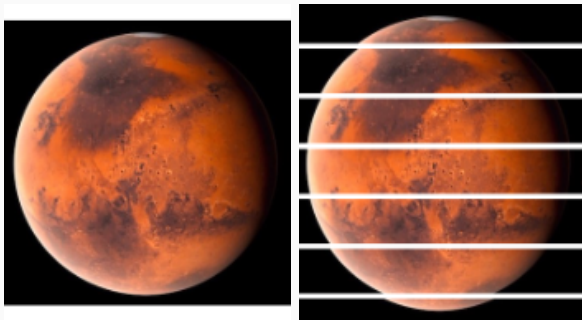
$$T_{scatter/gather} \approx \sum_{i=0}^{\log p - 1} (\alpha + \beta 2^i \frac{m}{p}) = \alpha \log p + \beta m (1 - \frac{1}{p})$$

$$T_{scatter/gather} = O(\alpha \log p + \beta m)$$

Particionamiento de datos: Procesamiento de imágenes

Ejemplo: **procesamiento de imágenes**

Solo las coordenadas del pixel son modificadas, independientemente de los pixels vecinos (desplazamiento, escala, rotación, etc)



Se toma especial cuidado en la distribución de la imagen entre procesos, normalmente se separa la imagen en regiones cuadradas, o en líneas y columnas.

Particionamiento de datos: Procesamiento de imágenes

Ejemplo: **Intensidad:**

Se aplica un cambio en la intensidad de cada pixel, independientemente en cada proceso



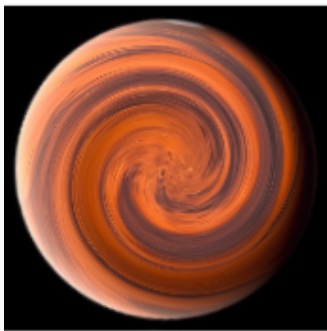
Particionamiento de datos: Procesamiento de imágenes

Ejemplo: **Image warping:**

Transformación de giro de una imagen con respecto a un centro de rotación (c_x, c_y)

$$x' = (x - c_x)\cos\theta + (y - c_y)\sin\theta + c_x$$

$$y' = (x - c_x)\sin\theta + (y - c_y)\cos\theta + c_y$$



Pseudocódigo paralelo (PRAM)

Algorithm 1: **Image-Warping**(I, n)

Entrada: Imagen de entrada I de tamaño $n \times n$

Salida: Imagen transformada O

```
1  $tx \leftarrow n/2$   $ty \leftarrow n/2$ ,  $DRAD \leftarrow 180/\pi$ 
2 for  $k \leftarrow 0$  to  $n^2 - 1$  pardo
3    $yp \leftarrow \lfloor k/n \rfloor$ 
4    $xp \leftarrow k \bmod n$ 
5    $radius \leftarrow \sqrt{(xp - tx)^2 + (yp - ty)^2}$ 
6    $theta \leftarrow (radius/2) \cdot DRAD$ 
7    $x \leftarrow \cos(theta)(xp - tx) - \sin(theta)(yp - ty) + tx$ 
8    $y \leftarrow \sin(theta)(xp - tx) + \cos(theta)(yp - ty) + ty$ 
9    $O[k] \leftarrow \text{BilinearInterpolate}(I, n, n, x, y)$ 
```

Tendremos una secuencia de cálculos de $n \times n$ pixels en tiempo

$$T_s = O(n^2)$$

En paralelo, para p procesos, el **tiempo de cómputo** es: $T_{comp} = O(\frac{n^2}{p})$

y el de **comunicación**: $T_{comm} = O(\alpha \log p + \beta \frac{n^2}{p})$

Por consiguiente: $T_p = T_{comm} + T_{comp} = O(\frac{n^2}{p} + \alpha \log p + \beta \frac{n^2}{p})$

Ya que la comunicación a los procesos y el retorno es lo que más cuesta, arquitectura de memoria compartida es óptima para este tipo de problemas (incluye GPUs)

Código: OpenMP - Image warping

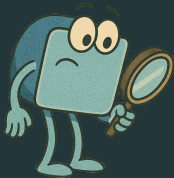
```
int index, xp,yp,tx=width/2, ty=height/2;
float x,y,radius,theta, PI=3.141527f, DRAD=180.0f/PI;

#pragma omp parallel for \
shared(inputImage, outputImage, width, height) \
private(x,y,index,.radius. theta,xp,yp)

for (yp=0;yp<height;yp++){
    for (xp=0;xp<width;xp++){
        index=xp+yp*width;
        radius=sqrtf((xp-tx)*(xp-tx)+(yp-ty)*(yp-ty));
        theta=(radius/2)*DRAD;
        x=cos(theta)*(xp-tx)-sin(theta)*(yp-ty)+tx;
        y=sin(theta)*(xp-tx)+cos(theta)*(yp-ty)+ty;
        outputImage[index]=\
            BilinearInterpolate(inputImage,width,height,x,
                                y);
    }
}
```

Haga un análisis de escalabilidad del algoritmo de Image-warping.

- en un paradigma compartido
- en un paradigma distribuido



Particionamiento de Tareas

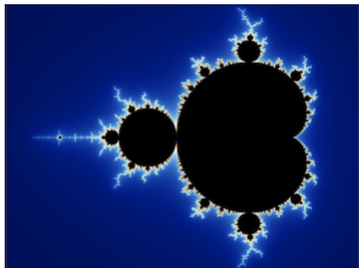
Paralelismo de tareas: Mandelbrot

Es un set de puntos complejos (imagen $n \times n$), que se calculan a través de la iteración de una función compleja no-divergente (iterada desde $z=0$) de forma

$$z_{k+1} = z_k^2 + c$$

donde z_{k+1} es la iteración (k+1) del número complejo $z = a + bi$ (con $i = \sqrt{-1}$), así como c es un número complejo. Se itera hasta

que la longitud del vector $z_{len} = \sqrt{z_{real}^2 + z_{imag}^2}$ es mayor que 2



Paralelismo de tareas: Mandelbrot

Algorithm 2: Mandelbrot secuencial

Entrada: $n_x, n_y, K_{\text{máx}}$

Salida: Imagen I

```
1 for  $i \leftarrow 0$  to  $n_y - 1$  do
2   for  $j \leftarrow 0$  to  $n_x - 1$  do
3      $c \leftarrow$  complejo asociado al píxel  $(i, j)$ 
4      $z \leftarrow 0, k \leftarrow 0$ 
5     while  $|z| \leq 2$  and  $k < K_{\text{máx}}$  do
6        $z \leftarrow z^2 + c$ 
7        $k \leftarrow k + 1$ 
8      $I[i, j] \leftarrow k$ 
```

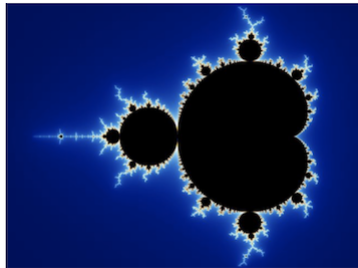
Paralelismo de tareas: Mandelbrot

En paralelo:

Cada pixel puede ser calculado independientemente del resto (el cálculo individual del pixel no es paralelizable)

Distribución estática de tareas:

Cada pixel necesita un número distinto de iteraciones, por lo que cada proceso recibe una porción de la imagen ($\frac{n^2}{p}$). Sin embargo, estas no terminan sincronizadas.



Paralelismo de tareas: Mandelbrot

Algorithm 3: **Mandelbrot paralelo**(n, K_{max})

Entrada: Imagen de $n \times n$ píxeles; máximo K_{max} de iteraciones

Salida: Matriz M

```
1 for  $(i, j) \in \{0, \dots, n-1\}^2$  pardo
2    $c \leftarrow$  complejo asociado al píxel  $(i, j)$ 
3    $z \leftarrow 0, k \leftarrow 0$ 
4   while  $|z| \leq 2$  and  $k < K_{max}$  do
5      $z \leftarrow z^2 + c$ 
6      $k \leftarrow k + 1$ 
7    $M[i, j] \leftarrow k$ 
```

Paralelismo de tareas: Mandelbrot

Para una imagen de $n \times n$ píxeles existen n^2 tareas independientes. Cada pixel realiza como máximo K_{max} iteraciones. Por tanto, el trabajo total es

$$W = O(n^2 K_{max}).$$

Como los n^2 píxeles pueden procesarse en paralelo, cada tarea realiza secuencialmente como máximo K_{max} iteraciones. Por tanto

$$T_{\infty} = O(K_{max}).$$

Aplicando el teorema de Brent para p procesos:

$$T_p = O\left(\frac{n^2 K_{max}}{p} + K_{max}\right)$$

Si K_{max} se considera constante, entonces para $p \leq n^2$, se puede simplificar a

$$T_p = O\left(\frac{n^2}{p}\right).$$

La comunicación consiste en recolectar los resultados:

$$T_{\text{comm}} = O(\log p(\alpha + \beta N^2))$$

donde α representa la latencia de comunicación y β el tiempo de transferencia por dato (ancho de banda)

Por tanto,

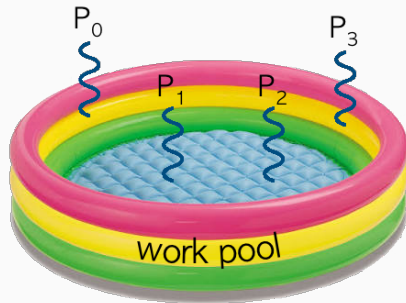
$$T_p = O\left(\frac{N^2}{p} + \log p(\alpha + \beta N^2)\right)$$

Nota: si considera el particionamiento inicial de datos (Scatter), la complejidad no cambia

Paralelismo de tareas: Mandelbrot

Distribución dinámica de tareas: Las tareas en paralelo se distribuyen en un área común (work pool), desde donde cada proceso obtiene una nueva tarea en caso quede inactivo

Aparece un overhead adicional por el paso de mensajes entre procesadores y el maestro (T_{msg}), así como el tiempo de planeamiento de tareas (T_{sched})



Paralelismo de tareas: Mandelbrot

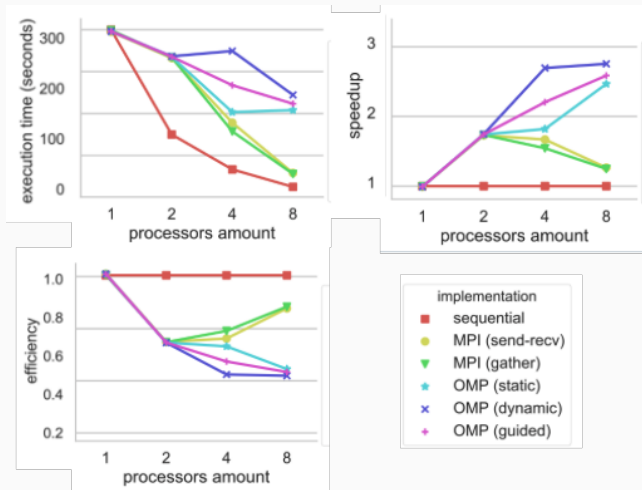
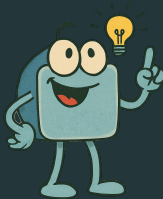


Figura 1: Comparación de métricas (MPI-OMP) en el problema de Mandelbrot ($K_{max} = 10^5$). De izquierda a derecha: T_{ejec} , S (speedup), E (eficiencia). Fuente: *Revista Innovación y Software. Vol 1. Set-Feb 2020: caso Mandelbrot*



Cuestionario

Join at
slido.com
#133 309



U2.6 Random en paralelo

Un algoritmo estocástico incorpora explícitamente variables aleatorias durante su ejecución. Para la misma entrada, dos ejecuciones pueden seguir caminos distintos o producir resultados ligeramente diferentes. Hay dos categorías:

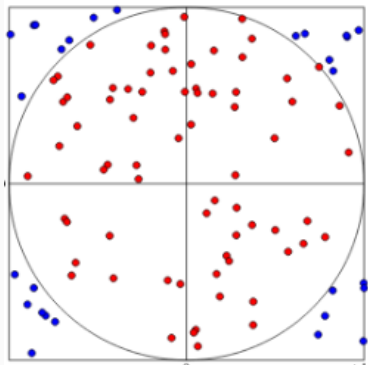
- Algoritmos **Monte Carlo**: el tiempo de ejecución está controlado, pero el resultado puede contener error.
- Algoritmos **Las Vegas**: el resultado es correcto, pero el tiempo de ejecución es una variable aleatoria.

Utilizan principios de cálculo probabilístico y estadística, para aproximar soluciones de problemas complejos. Por ello, los resultados son correctos con cierta probabilidad. Se utilizan en Física e ingeniería, Computación gráfica y diseño de productos, Finanzas, Estudio de sismos o fenómenos naturales, clima. Ejemplos:

- cálculo de π
- integración numérica
- random walk
- colisiones de partículas, riesgo financiero

Algoritmo Montecarlo para calculo de PI

- Genera N puntos random en el cuadrado exterior
- Determina el número de puntos n dentro del círculo
- Siendo
el área del círculo $\pi * r^2$,
y el área del cuadrado $4 * r^2$
- Entonces
 $n/N = \pi/4$, o $\pi = 4 * n/N$
- Generando un número suficiente de puntos se logra un resultado mas exacto
- $10^3 < N < 10^6$



Código: Algoritmo Montecarlo para calculo de PI (secuencial)

```
for(i=0; i<num_trials ; i++)
{
    x  = (double)rand() / RAND_MAX;
    y  = (double)rand() / RAND_MAX;
    test = x*x + y*y;
    if (test <= r*r) Ncirc++;
}

pi = 4.0 * ((double)Ncirc/((double)
    num_trials));
```

Código: Algoritmo Montecarlo para calculo de PI (OMP)

```
//Init Parallelization with reduction
#pragma omp parallel for reduction(+: count
    )
for(i=0; i<num_trials ; i++)
{
    x  = (double)rand() / RAND_MAX;
    y  = (double)rand() / RAND_MAX;
    test = x*x + y*y;
    if (test <= r*r) Ncirc++;
}

pi = 4.0 * ((double)Ncirc/((double)
    num_trials);
```

Algoritmo Montecarlo para calculo de PI

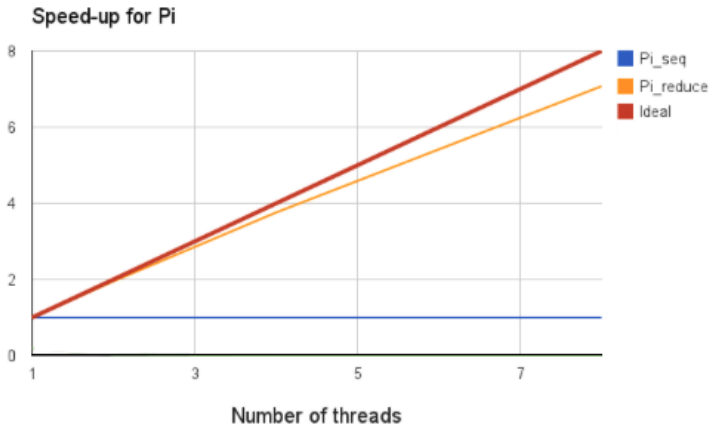


Figura 2: Speedup para el calculo de Pi

Integración numérica

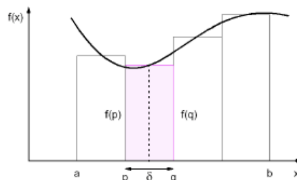
Dada una integral $I = \int_a^b f(x)dx$, ésta puede calcularse a través de la generación de N valores aleatorios entre a y b , y ser aproximados con $I_{MC} = \sum_N f(x)$, tal que cada una se aproxima con un rectángulo, asignado a cada proceso. Ya que las iteraciones para suma o integrales son independientes, se paralelizan fácilmente.

Método estático:

Cada área se calcula

con la fórmula $\frac{1}{2}(f(p) + f(q))\delta$.

Si se conoce δ de antemano, se comunica a cada proceso. Con p procesos, es $(b-a)/p$. Estas se ejecutarán en cada proceso hasta que el criterio de aproximación se cumpla. Luego se recopila y suma la información en el nodo maestro. La generación aleatoria puede delegarse a cada proceso, evitando así tiempos de comunicación.



Código: Integración numérica-OMP

```
wtime = omp_get_wtime();
total = 0.0;
# pragma omp parallel shared(a,b,n) private (i,x)
# pragma omp for reduction (+ : total)
for (int i = 0; i < n; i++) {
    x = ((double)(n-i-1)*a + (double)(i)*b) / (double)
        (n-1);
    total = total + f(x);
}
mtime = omp_get_wtime ( ) - wtime;
total = (b-a) * total / (double)n;
error = fabs (total - exact);
```

Generación random en paralelo

El generador 'linear-congruente' (LCG) produce números pseudo-aleatorios, utilizando la función $x_{i+1} = (ax_i + c) \bmod(m)$
Con a,c y m constantes, e.g. $a=16807$, $m=2^{31}-1$, $c=0$

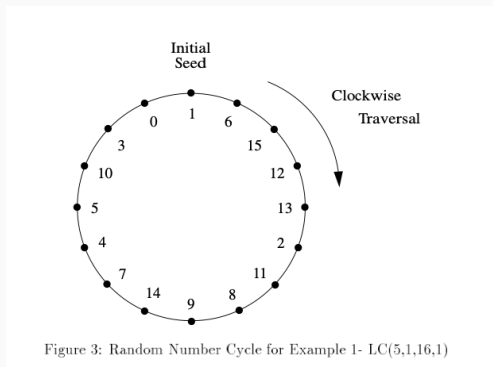


Figura 3: Ejemplo para $a=5$, $m=2^4$, $c=1$, $x_0 = 1$

- **Estrategia centralizada:** el maestro distribuye random a los procesos (bajo performance)
- **Estrategia replicativa:** cada proceso genera su propio seed, usando el mismo generador random (puede crear correlaciones)
- **Estrategia distribuída:** cada proceso genera su propio seed, basado en random de procesos vecinos (eficiente pero difícil de implementar)

Random tree en paralelo

Utiliza dos generadores (LGC), L y R

$$L(x) = x_L = a_L x + c_L \pmod{m}$$

$$R(x) = x_R = a_R x + c_R \pmod{m}$$

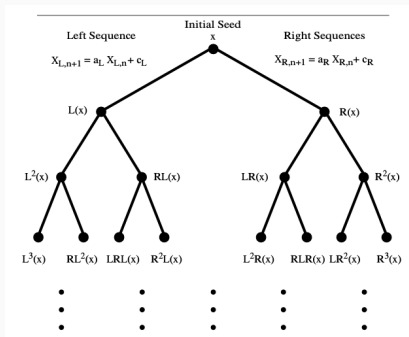


Figura 4: Operaciones del Random tree con respecto a x_0

RNG en paralelo

Implementa el generador recursivo múltiple MRG32k3a propuesto por L'Ecuyer (1999). Este provee de $1.8 \cdot 10^{19}$ random streams independientes, cada uno conteniendo $2.3 \cdot 10^{15}$ substreams. Cada substream tiene un período de $7.6 \cdot 10^{22}$. El periodo del generador en conjunto es de $3.1 \cdot 10^{57}$.

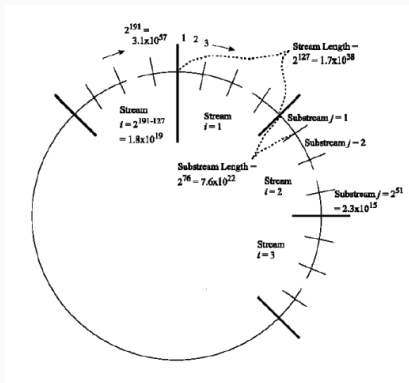


Figura 5: Streams y substreams

Performance random en paralelo

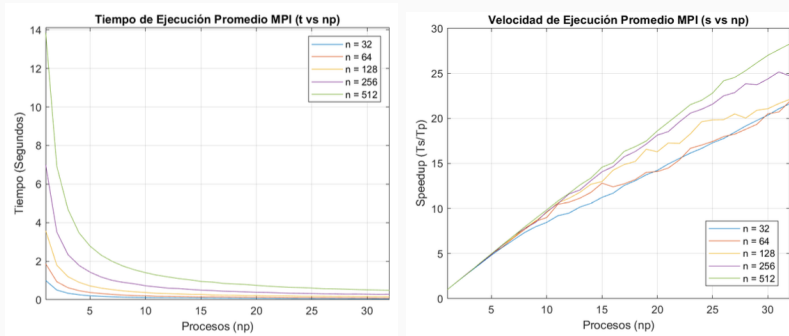


Figura 6: RNG: Tiempo vs procesos, Speedup vs procesos

Performance random en paralelo

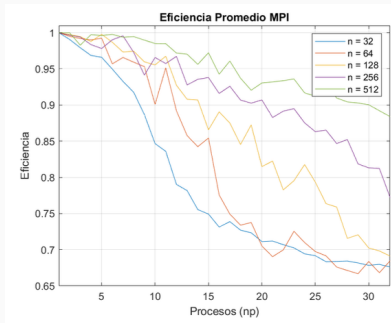


Figura 7: RNG: Eficiencia vs procesos

La granularidad describe cuánto trabajo computacional contiene una tarea paralela por cada unidad de overhead paralelo (sincronización o comunicación con otras tareas)

Se define como el ratio cómputo/comunicación

$$G = \frac{T_{comp}}{T_{comm}}$$

En el ejemplo de suma de numeros:

$$O\left(\frac{n^2/p}{\alpha \log p + \frac{\log p}{p} \beta n^2}\right)$$

De acuerdo a ello, la Granularidad se clasifica en:

- **Paralelismo fino:** el dominio esta particionado en un número grande de tareas, asignadas cada una a un proceso distinto. T_{comp} es bajo, lo que facilita el balance de cargas . Por otro lado, en este caso se necesitan mayor cantidad de procesos, lo que incrementa T_{comm}
- **Paralelismo grueso:** el dominio esta particionado en tareas largas, lo que implica un alto T_{comp} , lo que puede afectar el balanceo de cargas. Por otro lado, reduce T_{comm}
- **Paralelismo medio:** utiliza ambas formas en un mismo algoritmo. La mayoría de los casos aplicativos, usan esta forma de paralelismo



Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP*. 1st. McGraw-Hill Education Group, 2003. isbn: 0071232656.



Peter S. Pacheco. *An Introduction to Parallel Programming*. 1st. Morgan Kaufmann, 2011. isbn: 978-0-12-374260- 5.

